

Final Project

Build Real Science with AI

Why This Project

In this project, you will use AI to build a piece of scientific software that is bigger than anything you could make by hand this semester, and you will finish with a greatly improved understanding of both the science and the computation.

The homeworks are short on purpose, because each one tests you on a single scientific and computational idea. The project is where you choose your own question and go deep. You will complete it on your own, with Claude as your collaborator and a TA as your coach.

The project has two goals:

1. **Grow your understanding of a scientific problem.** By December, you will be able to teach your scientific problem to a friend who has never heard of it.
2. **Grow your understanding of the computational approach that solves it.** You should be able to explain what your program does, why it works, what assumptions it makes, and how you know that its output is right.

AI makes it easy to finish a project and learn nothing. We are not grading a project; rather, we grade whether your brain got bigger along the way, whether you can defend what you built, whether you checked it to make sure it's right, and whether you used AI to learn rather than to avoid learning.

This is how professional scientists and engineers already work outside this classroom. The person who directs the AI should own the result, including its mistakes, in the same way that an ancient general would either win a battle or be carried out on his shield. Your job is to be the manager: establish the questions, decide what counts as done, check the work, refine repeatedly, and understand it well enough to stand behind it.

Project Components

The project is worth 20% of your course grade. The components are below, each with its share of the project grade.

Component	What it is	Share
Proposal	One page answering the three questions below	5%
Build journal	Your Claude Code logs and git history, read at four checkpoints	35%
TA check-in	A 15-minute conversation with your assigned TA	5%
Final app and code	Your program, live on the web, with its repository	10%
Video demo	At most 5 minutes, your program answering your question	10%
Project assessment	Closed and in class, on the final day of the course	35%

TA check-in

Between Wednesday, November 4 and Friday, November 13, you will meet for 15 minutes with your assigned TA, who will stay your TA for the rest of the project. Bring your laptop and be ready to explain your question, show your program running, and answer questions about your model and your algorithm. This is a coaching conversation: you earn full credit by showing up prepared with something that runs and giving honest answers, including “I don’t know yet.” The point is to catch problems while there is still time to fix them.

Final app and code

Your final program must be **deployed to the web**, so that anyone with the link can use it without installing anything. (We will show you how.) Your repository should include a README that explains what the program does and how to run it.

Video demo

A screen recording of at most 5 minutes, narrated in your own voice, linked from your README and your website. Show us your question, your program answering it, and how you know that the answer is right.

Demo day

The next-to-last class meeting is demo day, which runs like a science fair. Half the class sets up at tables with laptops while the other half walks around, and then we swap. Demo day is not graded. Treat it as practice for the project assessment two days later: explaining your project out loud and answering questions about it is the fastest way to find what you do not yet understand.

Project assessment

The final class meeting is a closed assessment, on paper, about your own project: the science behind it, the computation inside it, and what you learned by building it. It will also include a short reflective essay about your experience.

Choosing a Project

Your project must be inspired by science: a computational model of a real scientific phenomenon that answers the three questions below. You are not expected to do novel research. **Our suggested direction is to reproduce a published scientific result, and then take it wherever your interest leads.** Reproducing a well-known result, and understanding why it holds, is already a serious piece of science that gives you something to check against; what you do with it next is where the project becomes yours. A model of some other phenomenon that fascinates you is also welcome, as long as you can answer the third question.

Your proposal answers three questions

1. **What is your scientific question?**
2. **What will you build to answer it?** A working model that lets someone explore your question, not a single figure.
3. **How will you check your model?** Name at least one concrete check, such as a published value your model should reproduce, a special case you can solve by hand, or a behavior the science says your model must show. “It runs without crashing” is not a check.

Your proposal (at most one page) also names the paper or phenomenon that inspires your project, the data you will use if any, and, if you start from a paper, where you want to take it. **If your project uses data, you must have that data in hand when you propose**, meaning downloaded and in your repository. Data that is “available on request,” or that sits behind a government or institutional portal you have not yet downloaded from, does not count. You will get feedback before Fall Break, so that you can start building over the break if you like. If your proposal has a problem, you will have one opportunity to revise it.

Where to look for ideas

The project is deliberately open-ended, and we encourage creativity. For inspiration, see the **Project Ring of Honor**, which collects notable projects from previous runs of the course. Those projects were built before AI, by teams, in a different language; with Claude at your side, you should aim at least that high on your own.

Projects we will not accept

- A project with no scientific question, such as a video game or a dashboard.
- Reinventing a standard package, such as rewriting functions from math or numpy.
- A pipeline that feeds one library call into the next without solving a problem.
- A project whose only check is that it runs.

Pitfalls we see every year

- **The data never arrives.** Requests for data go unanswered, and portals often turn away undergraduates. This is why we require data in hand.
- **The paper is obscure.** If you start from a paper, a five-year-old paper with two citations is rarely worth reproducing. An older, famous result that clearly made an impact is more rewarding and gives you more to check against.
- **The scope is too big.** Get the smallest version of your model working and checked first, then add to it. Claude makes it tempting to build everything at once; resist.

How You Will Work: The Build Journal

Your build journal is the record of how you worked and whether your brain grew. It is one of the two largest parts of your grade.

Everything happens in Claude Code. Your whole project lives in one GitHub repository, and you do all of your project work in Claude Code inside it, just as we did in class with spatial games. That includes learning the science: keep your paper or other sources in the repository and have Claude teach them to you, quiz you, and work out with you what your model assumes. Commit early and often, with commit messages in your own words.

What you submit. We will give you a one-line command that copies your Claude Code session logs into a `journal/` folder in your repository. Together with your git history, those logs are your journal.

How it is read. We read your journal at the four checkpoints listed in the timeline below; you do not need to do anything on those dates except keep your repository up to date.

What we look for.

- **Steady work.** Sessions spread across the weeks rather than one burst at the end. Not every day; a rhythm.
- **Learning with AI.** You had Claude teach you the science and quiz you, you asked why, and you explained ideas back in your own words. After Claude writes code, you ask it to explain that code; when something breaks, you debug it together rather than handing it the bug.

- **Checking the work.** You asked for tests and compared your output against the check from your proposal.

What we do not grade. The number of prompts, how polished they are, or whether you let Claude write most of the code. Dead ends are a good thing.

Privacy. Only your project repository is collected, never the rest of your Claude account, in case you are wondering about r/AIGirlfriend. You may redact anything personal that slips in before you submit. Only course staff read your journal.

Important Dates

All deadlines are at 11:59 pm Eastern. Journal checkpoints need nothing from you except an up-to-date repository. Nothing is due during Fall Break or Thanksgiving Break.

Date	What happens
Wednesday, September 30	Project released
Wednesday, October 7	Proposal due
Friday, October 9	Staff feedback on proposals
Wednesday, October 21	Journal checkpoint 1; revised proposals due (only if we ask for one)
Friday, October 30	Journal checkpoint 2; project office hours
November 4 to 13	TA check-ins
Friday, November 20	Journal checkpoint 3; first version of your app live on the web
Tuesday, December 1	Demo day; final app, code, and video due ; last journal checkpoint
Thursday, December 3	Project assessment , in class

Getting Help

Ask project questions in the Project category on Ed, and come to office hours, including the project office hours on October 30. Your assigned TA is your first stop once the check-ins begin.

To help with your written and oral communication, the university has a central service called the Global Communication Center (GCC). You should make use of their services throughout your time at CMU; see <https://www.cmu.edu/gcc/>.